



A P P L I C A T I O N W E B T R A I T E U R
É V É N E M E N T I E L

VITE & GOURMAND

Documentation de déploiement

Application Web Traiteur Événementiel

Projet réalisé dans le cadre du titre professionnel
Développeur Web et Web Mobile

· Projet : Vite & Gourmand

Table des matières

Architecture technique	3
Flux de données	3
Prérequis	4
Développement (avec Docker)	4
Production	4
Installation locale (développement)	5
3.1 Cloner le dépôt	5
3.2 Configurer l'environnement	5
3.3 Lancer avec Docker Compose	5
3.4 Initialiser les données	5
3.5 Vérifier le bon fonctionnement	5
3.6 Compte admin par défaut	5
Configuration	6
PostgreSQL	6
MongoDB	6
Backend (API)	6
Email (SMTP)	6
Frontend / CORS	7
Paiement / Acompte (devis)	7
IA / LLM (suggestions et chatbot)	7
Base de données	8
5.1 Schéma PostgreSQL	8
5.2 Réinitialiser la base	8
5.3 MongoDB (analytiques)	8
Docker – détail des services	9
6.1 Frontend (développement)	9
6.2 Backend API (développement)	9
6.3 Frontend (production)	9
6.4 Backend API (production)	9
6.5 Healthchecks	10
6.6 Ordre de démarrage	10
6.7 Volumes persistants	10
Déploiement en production	11
7.1 Option A – VPS (serveur dédié)	11
7.2 Option B – Docker en production	12
7.3 Option C – PaaS (Render, Railway, Fly.io)	12
Sécurité	13
8.1 Mesures implémentées	13
8.2 Checklist production	13
Tests post-déploiement	14
9.1 Vérifications fonctionnelles	14
9.2 Vérifications techniques	14
9.3 Tests unitaires (Jest)	14
Maintenance et opérations	15
10.1 Logs	15
10.2 Sauvegardes	15
10.3 Mises à jour	15
10.4 Commandes utiles	15

SECTION 01

Architecture technique

L'application est entièrement conteneurisée via Docker Compose. Cinq services collaborent pour former l'environnement complet :

SERVICE	IMAGE	RÔLE	PORT
vg_frontend	Node 20 Alpine + Vite	Application React (SPA)	5173
vg_api	Node 20 Alpine	API REST Express.js	3000
vg_postgres	PostgreSQL 16 Alpine	Base de données relationnelle	5432
vg_mongo	MongoDB 7	Analytiques (logs, statistiques)	27017
vg_mailhog	MailHog	Interception email en développement	1025/8025

Flux de données

- Le frontend communique avec l'API via VITE_API_URL (HTTP REST + JWT)
- L'API utilise PostgreSQL comme base principale (requêtes SQL via pg)
- L'API utilise MongoDB pour les analytiques (Mongoose)
- L'API envoie les emails via SMTP (MailHog en dev, SMTP réel en prod)
- L'API intègre un module IA via API LLM externe (Groq/OpenAI)

SECTION 02

Prérequis

Développement (avec Docker)

OUTIL	VERSION MINIMALE	VÉRIFICATION
Docker	24+	<code>docker --version</code>
Docker Compose	v2+	<code>docker compose version</code>
Git	2.x	<code>git --version</code>

Production

OUTIL	VERSION MINIMALE
Node.js	20 LTS
PostgreSQL	16
MongoDB	7
Nginx	dernière stable
Serveur SMTP	(Brevo, SendGrid, etc.)

SECTION 03

Installation locale (développement)

3.1 Cloner le dépôt

```
git clone <url-du-repo>
cd vite-gourmand
```

3.2 Configurer l'environnement

```
cp .env.example .env
```

Modifier les valeurs dans .env (voir section 4).

3.3 Lancer avec Docker Compose

```
docker compose up --build -d
```

Cette commande construit les images, démarre PostgreSQL et attend le healthcheck, exécute les scripts SQL d'initialisation (schema → seed → migrations), puis démarre MongoDB, MailHog, l'API et le frontend dans le bon ordre.

3.4 Initialiser les données

```
# Seed PostgreSQL (admin + données de démo)
docker exec vg_api npm run seed:postgres

# Seed MongoDB (analytiques)
docker exec vg_api npm run seed:mongo

# Ou les deux en une fois
docker exec vg_api npm run seed
```

3.5 Vérifier le bon fonctionnement

SERVICE	URL	ATTENDU
Frontend	http://localhost:5173	Page d'accueil Vite & Gourmand
API Health	http://localhost:3000/api/health	{ "status": "ok" }
MailHog UI	http://localhost:8025	Interface de test email

3.6 Compte admin par défaut

CHAMP	VALEUR
Email	admin@vitegourmand.fr
Mot de passe	Admin123!@#

Important : Changer ce mot de passe immédiatement en production.

SECTION 04

Configuration

Variables d'environnement à définir dans le fichier `.env` (copié depuis `.env.example`).

PostgreSQL

VARIABLE	DESCRIPTION	EXEMPLE
POSTGRES_HOST	Hôte PostgreSQL	db_sql / localhost
POSTGRES_PORT	Port	5432
POSTGRES_DB	Nom de la base	vitegourmand
POSTGRES_USER	Utilisateur	vitegourmand
POSTGRES_PASSWORD	Mot de passe	(générer un mot de passe fort)

MongoDB

VARIABLE	DESCRIPTION	EXEMPLE
MONGO_HOST	Hôte MongoDB	db_mongo / localhost
MONGO_PORT	Port	27017
MONGO_DB	Nom de la base	vitegourmand_analytics
MONGO_USER	Utilisateur	vitegourmand
MONGO_PASSWORD	Mot de passe	(générer un mot de passe fort)

Backend (API)

VARIABLE	DESCRIPTION	EXEMPLE
NODE_ENV	Environnement	development / production
PORT	Port de l'API	3000
JWT_SECRET	Clé secrète JWT (≥32 chars)	(openssl rand -hex 32)
JWT_REFRESH_SECRET	Clé secrète refresh token	(openssl rand -hex 32)
JWT_EXPIRY	Durée access token	15m
JWT_REFRESH_EXPIRY	Durée refresh token	7d

Email (SMTP)

VARIABLE	DESCRIPTION	DEV	PROD
SMTP_HOST	Serveur SMTP	mailhog	smtp.brevo.com
SMTP_PORT	Port SMTP	1025	587
SMTP_FROM	Adresse expéditeur	contact@vitegourmand.fr	idem
SMTP_USER	Identifiant SMTP	(vide)	(clé API)
SMTP_PASS	Mot de passe SMTP	(vide)	(clé API)

Frontend / CORS

VARIABLE	DESCRIPTION	EXEMPLE
VITE_API_URL	URL de l'API pour le frontend	http://localhost:3000/api
FRONTEND_URL	URL du frontend (emails)	http://localhost:5173
FRONTEND_URLS	Origines CORS autorisées (virgule)	http://localhost:5173

Paieement / Acompte (devis)

VARIABLE	DESCRIPTION	EXEMPLE
BANK_ACCOUNT_NAME	Nom du titulaire du compte	(nom entreprise)
BANK_IBAN	IBAN pour virement	(IBAN)
BANK_BIC	Code BIC	(BIC)
ON_SITE_ADDRESS	Adresse pour paiement sur place	(adresse)
ON_SITE_HOURS	Horaires d'accueil	(horaires)
ON_SITE_PHONE	Téléphone	(téléphone)

IA / LLM (suggestions et chatbot)

VARIABLE	DESCRIPTION	EXEMPLE
LLM_BASE_URL	URL de l'API LLM	https://api.groq.com/openai/v1
LLM_MODEL	Modèle LLM	llama-3.3-70b-versatile
OPENAI_API_KEY	Clé API (Groq ou OpenAI)	(clé API)

Le module IA est utilisé pour les suggestions de menus, l'estimation budgétaire et le chatbot d'assistance. La clé OPENAI_API_KEY fonctionne avec l'API compatible OpenAI de Groq.

SECTION 05

Base de données

5.1 Schéma PostgreSQL

Les scripts SQL sont exécutés automatiquement au premier démarrage du conteneur PostgreSQL, dans l'ordre :

ORDRE	FICHIER	CONTENU
01	database/schema.sql	Tables principales : users, menus, dishes, orders, reviews, etc.
02	database/seed.sql	Données de démonstration (3 users, 8 menus, 40 plats, horaires, pages légales)
03	database/quotes_migration.sql	Module devis (quotes, quote_items, quote_options, quote_status_history) + 20 options préchargées
04	database/payments_migration.sql	Colonnes paiement sur orders (payment_method, payment_status, deposit_amount, quote_id)
05	database/quotes-deposit-request.sql	Colonnes suivi envoi instructions acompte sur quotes
06	database/06-direct-order-deposits.sql	Ajout statuts deposit_pending et confirmed à l'enum order_status

5.2 Réinitialiser la base

```
# Arrêter et supprimer le volume PostgreSQL
docker compose down
docker volume rm vite-gourmand_pg_data

# Relancer (réexécute les scripts d'init)
docker compose up -d

# Re-seeder
docker exec vg_api npm run seed
```

5.3 MongoDB (analytiques)

MongoDB stocke les données analytiques (statistiques, logs de visite). Ces données sont non critiques et peuvent être recrées via :

```
docker exec vg_api npm run seed:mongo
```


SECTION 06

Docker — détail des services

6.1 Frontend (développement)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY index.html vite.config.js ./
EXPOSE 5173
CMD ["npx", "vite", "--host", "0.0.0.0"]
```

- Bind mounts : src/ et public/ sont montés en volume → hot reload
- Volume nommé : frontend_node_modules → isole les node_modules du host

6.2 Backend API (développement)

```
FROM node:20-alpine
RUN apk add --no-cache python3 make g++ # pour bcrypt natif
WORKDIR /app
COPY package*.json ./
RUN npm install
EXPOSE 3000
CMD ["npm", "run", "dev"] # node --watch src/index.js
```

- Bind mount : src/ monté → rechargement automatique (Node --watch)
- Volume nommé : api_node_modules → isole les node_modules

6.3 Frontend (production)

```
# Build multi-stage
FROM node:20-alpine AS build
WORKDIR /app
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- Build multi-stage : image finale ~30 Mo (Nginx Alpine)
- VITE_API_URL injecté au build via ARG

6.4 Backend API (production)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/index.js"]
```

- npm ci --omit=dev : n'installe que les dépendances de production
- Pas de --watch, exécution directe

6.5 Healthchecks

SERVICE	COMMANDE	INTERVALLE
PostgreSQL	<code>pg_isready -U \$POSTGRES_USER -d \$POSTGRES_DB</code>	5s (20 retries)
MongoDB	<code>mongosh --eval "db.adminCommand('ping')"</code>	5s (10 retries)
API	<code>HTTP GET /api/health (via Node)</code>	10s (10 retries)

6.6 Ordre de démarrage

```
vg_postgres --(healthy)→ vg_api --(healthy)→ vg_frontend  
vg_mongo    --(healthy)→ vg_api  
vg_mailhog  --(started)→ vg_api
```

6.7 Volumes persistants

VOLUME	USAGE
<code>pg_data</code>	Données PostgreSQL
<code>mongo_data</code>	Données MongoDB
<code>api_node_modules</code>	Dépendances backend (isolées du host)
<code>frontend_node_modules</code>	Dépendances frontend (isolées du host)

Astuce : Après ajout d'une dépendance npm, exécuter : `docker exec vg_api npm install && docker restart vg_api` (ou `docker exec vg_frontend npm install && docker restart vg_frontend` pour le frontend).

SECTION 08

SECTION 07

Déploiement en production

Option A — VPS (serveur dédié)

Architecture classique : Nginx reverse proxy + SSL, fichiers statiques depuis dist/, API Node.js via PM2. Adapté pour un contrôle total du serveur (Ubuntu 22.04 LTS, 2 Go RAM minimum).

Option B — Docker en production

Utilisation des Dockerfiles de production (multi-stage pour le frontend, npm ci --omit=dev pour le backend). Lancement via docker-compose.prod.yml.

Option C — Déploiement cloud actuel (Vercel + Render + Neon + Atlas + Brevo)

Configuration utilisée en production. L'application est déployée dans le cloud avec des services managés, sans serveur à administrer.

Architecture cloud de production

Frontend (React + Vite) → Vercel — <https://vite-gourmand-three.vercel.app>

Backend (Node.js / Express) → Render (Web Service) — <https://vite-gourmand.onrender.com>

PostgreSQL → Neon (serverless PostgreSQL 16 managé)

MongoDB → MongoDB Atlas (cluster M0) — analytiques et statistiques

Emails → Brevo (API HTTP) — emails transactionnels

Frontend – Vercel

Le frontend React est déployé sur Vercel. Déploiement automatique à chaque push sur main via intégration GitHub.

Framework Preset : Vite

Build Command : npm run build

Output Directory : dist

Root Directory : frontend

Variable : VITE_API_URL = <https://vite-gourmand.onrender.com/api>

Fichier vercel.json : rewrites SPA (toutes les routes vers index.html)

Backend – Render

L'API Express est déployée comme Web Service sur Render. Déploiement automatique sur push.

Runtime : Node 20

Build Command : npm ci --omit=dev

Start Command : node src/index.js

Root Directory : backend

Variables d'environnement sur Render :

NODE_ENV = production

DATABASE_URL = postgresql://...@ep-xxx.neon.tech/dbname?sslmode=require

MONGO_URI = mongodb+srv://...@cluster.mongodb.net/vitegourmand_analytics

JWT_SECRET / JWT_REFRESH_SECRET = clés générées via openssl rand -hex 32

BREVO_API_KEY = clé API Brevo

SMTP_FROM = contact@vitegourmand.fr

FRONTEND_URL = <https://vite-gourmand-three.vercel.app>

FRONTEND_URLS = <https://vite-gourmand-three.vercel.app> (CORS)

ADMIN_EMAIL = admin@vitegourmand.fr

LLM_BASE_URL / OPENAI_API_KEY = configuration IA (Grok)

SECTION 08

Note : Render bloque les connexions SMTP sortantes. L'envoi d'emails passe par l'API HTTP de Brevo.

PostgreSQL – Neon

Neon fournit PostgreSQL 16 serverless avec SSL obligatoire. Contient toutes les tables métier.

Connexion via DATABASE_URL (format postgresql://)

SSL activé par défaut (sslmode=require)

Migrations SQL via la console SQL de Neon

Sauvegardes automatiques (branching et point-in-time recovery)

MongoDB – Atlas

Cluster M0 (gratuit) pour les données analytiques.

Connexion via MONGO_URI (format mongodb+srv://)

Network Access : 0.0.0.0/0 (Render n'a pas d'IP fixe)

Base : vitegourmand_analytics

Emails – Brevo (API HTTP)

Brevo gère tous les emails transactionnels : confirmation commande, bienvenue, reset mot de passe, notification réponse contact.

API HTTP (pas SMTP, bloqué par Render)

Expéditeur : contact@vitegourmand.fr

Templates HTML dans emailService.js

Workflow de déploiement (CI/CD)

Le déploiement est automatisé via l'intégration Git :

1. Le développeur pousse sur main (git push origin main)
2. Vercel rebuild automatiquement le frontend (~1 min)
3. Render redéploie automatiquement le backend (~2-3 min)
4. Migrations BDD : appliquer le SQL via Neon avant le push

URLs de production

Site web : <https://vite-gourmand-three.vercel.app>

API : <https://vite-gourmand.onrender.com/api>

Health check : <https://vite-gourmand.onrender.com/api/health>

SECTION 08

Sécurité

8.1 Mesures implémentées

MESURE	DÉTAIL
Helmet.js	Headers HTTP sécurisés (CSP, HSTS, X-Frame-Options, etc.)
CORS	Origines restreintes via FRONTEND_URLS
Rate limiting	200 req/15min (général), 20 req/15min (auth), 30 req/15min (AI)
JWT	Access token 15min + refresh token 7 jours
bcrypt	Hash des mots de passe, 12 salt rounds
express-validator	Validation de toutes les entrées utilisateur
Mots de passe	≥10 caractères, majuscule + minuscule + chiffre + caractère spécial
Reset tokens	crypto.randomBytes(32), expiration 1h, usage unique
RGPD	Consentement à l'inscription, export données (Art. 20), anonymisation (Art. 17)
Pino logger	JSON structuré en production, format lisible en développement

8.2 Checklist production

- Changer le mot de passe admin par défaut
- Générer des secrets JWT forts (openssl rand -hex 32)
- Mots de passe PostgreSQL et MongoDB forts
- Définir NODE_ENV=production
- HTTPS obligatoire (redirection HTTP → HTTPS)
- CORS : uniquement le domaine de production
- SMTP : serveur email de production (pas MailHog)
- Désactiver les ports PostgreSQL/MongoDB exposés sur Internet
- Sauvegardes automatiques de la base de données
- Configurer la clé API LLM (OPENAI_API_KEY) pour le module IA

SECTION 09

Tests post-déploiement

9.1 Vérifications fonctionnelles

TEST	ENDPOINT / ACTION	RÉSULTAT ATTENDU
Health API	GET /api/health	200 { "status": "ok" }
Page d'accueil	/	Affichage correct
Inscription	Créer un compte	Email de bienvenue reçu
Connexion	Login avec email/mdp	JWT retourné, dashboard accessible
Consulter menus	GET /api/menus	Liste des menus actifs
Passer commande	Commander un menu	Commande créée, email de confirmation
Créer devis	POST /api/quotes	Devis créé en statut draft
Reset password	Mot de passe oublié	Email avec lien reçu, MDP modifiable
Admin	Se connecter en admin	Dashboard admin accessible
Analytics	GET /api/analytics/*	Données analytiques retournées
Suggestions IA	GET /api/suggestions/menus	Suggestions de menus retournées
CORS	Requête cross-origin non autorisée	Rejetée (403)

9.2 Vérifications techniques

```
# Vérifier que l'API répond
curl -s https://votredomaine.fr/api/health | jq

# Vérifier les headers de sécurité
curl -I https://votredomaine.fr

# Vérifier le SSL
openssl s_client -connect votredomaine.fr:443 -brief

# Vérifier les logs
docker logs vg_api --tail 50
# ou
pm2 logs vg-api --lines 50
```

9.3 Tests unitaires (Jest)

Le backend inclut des tests Jest pour les services et repositories :

```
# Lancer les tests dans le conteneur
docker exec vg_api npm test

# Ou en local
cd backend && npm test
```

SECTION 10

Maintenance et opérations

10.1 Logs

```
# Docker
docker logs vg_api -f          # API (temps réel)
docker logs vg_postgres -f     # PostgreSQL

# PM2 (production VPS)
pm2 logs vg-api
```

L'API utilise Pino comme logger (JSON structuré en production, format lisible en développement).

10.2 Sauvegardes

```
# Backup PostgreSQL
docker exec vg_postgres pg_dump -U vitegourmand vitegourmand > backup_$(date +%Y%m%d).sql

# Restaurer
cat backup_20260307.sql | docker exec -i vg_postgres psql -U vitegourmand -d vitegourmand

# Backup MongoDB
docker exec vg_mongo mongodump --out /data/backup --authenticationDatabase admin -u vitegourmand -p <password>
```

10.3 Mises à jour

```
# Récupérer les dernières modifications
git pull origin main

# Reconstruire et relancer (Docker)
docker compose up --build -d

# Ou en production VPS
cd /var/www/vite-gourmand
git pull
cd backend && npm ci --omit=dev
cd ../frontend && npm ci && npm run build
pm2 restart vg-api
```

10.4 Commandes utiles

```
# Accéder à la DB PostgreSQL
docker exec -it vg_postgres psql -U vitegourmand -d vitegourmand

# Accéder au shell MongoDB
docker exec -it vg_mongo mongosh -u vitegourmand -p <password>

# Recréer les node_modules après ajout de dépendance
docker exec vg_api npm install && docker restart vg_api
docker exec vg_frontend npm install && docker restart vg_frontend

# Vérifier l'état des conteneurs
docker compose ps

# Voir l'utilisation des ressources
docker stats
```



T R A I T E U R É V É N E M E N T I E L

VITE & GOURMAND

MCD — Modèle Conceptuel de Données

Projet : Vite & Gourmand
Session : 2026

Table des matières

Liste des entités.....	3
Attributs principaux.....	4
UTILISATEUR.....	4
MENU.....	5
PLAT.....	5
IMAGE_MENU.....	5
COMMANDE.....	6
AVIS.....	7
DEVIS.....	7
LIGNE_DEVIS.....	8
OPTION_DEVIS.....	8
HORAIRE.....	8
MESSAGE_CONTACT.....	9
PAGE_LEGALE.....	9
Associations et cardinalités.....	10
Cardinalités – notation synthétique.....	11
Relations principales.....	11
Relations optionnelles.....	11
Règles de gestion.....	12
RG-01 – Gestion des rôles.....	12
RG-02 – Composition des menus.....	12
RG-03 – Commande liée à un menu.....	12
RG-04 – Calcul du prix de commande.....	12
RG-05 – Remise groupe.....	12
RG-06 – Gestion du stock.....	12
RG-07 – Cycle de vie de la commande.....	12
RG-08 – Acompte obligatoire.....	12
RG-09 – Avis après commande.....	12
RG-10 – Note de 1 à 5.....	12
RG-11 – Structure d'un devis.....	13
RG-12 – Cycle de vie du devis.....	13
RG-13 – Conversion devis → commande.....	13
RG-14 – Date de validité du devis.....	13
RG-15 – Types de plats.....	13
RG-16 – Unités de tarification des options.....	13
RG-17 – Consentement RGPD.....	13
RG-18 – Unicité des horaires.....	13
RG-19 – Pages légales.....	13
RG-20 – Assignment des devis.....	13
RG-21 – Scoring client.....	13
RG-22 – Mots de passe.....	13
Correspondance MCD / Tables SQL.....	14
Tables exclues du MCD (techniques / historique).....	14
Corrections apportées (v3).....	15

SECTION 01

Liste des entités

Le MCD comporte 12 entités métier, regroupées en 4 familles selon leur rôle dans le système.

ENTITÉS CATALOGUE — BLEU

ENTITÉ	DESCRIPTION	TABLE SQL
UTILISATEUR	Client, employé ou administrateur	users
MENU	Menu proposé au catalogue	menus
PLAT	Plat composant un menu	dishes
IMAGE_MENU	Photo de la galerie d'un menu	menu_images

ENTITÉS TRANSACTIONNELLES — VERT

ENTITÉ	DESCRIPTION	TABLE SQL
COMMANDE	Commande passée par un client	orders
AVIS	Avis déposé après une commande	reviews

ENTITÉS DEVIS — JAUNE

ENTITÉ	DESCRIPTION	TABLE SQL
DEVIS	Demande de devis pour un événement	quotes
LIGNE_DEVIS	Ligne de détail d'un devis	quote_items
OPTION_DEVIS	Prestation additionnelle au catalogue	quote_options

ENTITÉS AUTONOMES — VIOLET

ENTITÉ	DESCRIPTION	TABLE SQL
HORAIRE	Horaires d'ouverture par jour	business_hours
MESSAGE_CONTACT	Message envoyé via le formulaire de contact	contact_messages
PAGE_LEGALE	Page légale éditée (mentions, confidentialité, CGV)	legal_pages

SECTION 02

Attributs principaux

UTILISATEUR

ATTRIBUT	TYPE	CONTRAINTE
<i>id_utilisateur</i>	UUID	Identifiant (PK)
nom	VARCHAR(100)	NOT NULL
prenom	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
telephone	VARCHAR(20)	NOT NULL
adresse	TEXT	NOT NULL
pays	VARCHAR(100)	Optionnel
role	ENUM	{user, employee, admin}
statut	ENUM	{active, inactive}
mot_de_passe_hash	VARCHAR(255)	NOT NULL
token_reinitialisation	VARCHAR(255)	Optionnel
expiration_token	TIMESTAMP	Optionnel
consentement_rgpd	BOOLEAN	NOT NULL
date_consentement_rgpd	TIMESTAMP	Optionnel
date_anonymisation	TIMESTAMP	Optionnel (RGPD Art. 17)
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

MENU

ATTRIBUT	TYPE	CONTRAINTE
<i>id_menu</i>	UUID	Identifiant (PK)
titre	VARCHAR(255)	NOT NULL
description	TEXT	NOT NULL
theme	VARCHAR(100)	NOT NULL
regime	VARCHAR(100)	NOT NULL
prix_minimum	DECIMAL(10,2)	>= 0
nb_personnes_min	INT	> 0
stock	INT	>= 0
conditions	TEXT	Optionnel
image_url	VARCHAR(500)	Optionnel (image principale)
actif	BOOLEAN	NOT NULL
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

PLAT

ATTRIBUT	TYPE	CONTRAINTE
<i>id_plat</i>	UUID	Identifiant (PK)
nom	VARCHAR(255)	NOT NULL
description	TEXT	Optionnel
type	ENUM	{entree, plat, dessert}
allergenes	TEXT[]	Tableau de texte
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

IMAGE_MENU

ATTRIBUT	TYPE	CONTRAINTE
<i>id_image</i>	UUID	Identifiant (PK)
url	VARCHAR(500)	NOT NULL
position	INT	Ordre d'affichage

COMMANDE

ATTRIBUT	TYPE	CONTRAINTE
id_commande	UUID	Identifiant (PK)
nb_personnes	INT	> 0
adresse_livraison	TEXT	NOT NULL
ville_livraison	VARCHAR(255)	NOT NULL
date_livraison	DATE	NOT NULL
heure_livraison	TIME	NOT NULL
distance_km	DECIMAL(10,2)	>= 0 (calcul livraison)
prix_menu	DECIMAL(10,2)	>= 0 (prix unitaire)
prix_livraison	DECIMAL(10,2)	>= 0 (frais livraison)
remise	DECIMAL(10,2)	>= 0
prix_total	DECIMAL(10,2)	>= 0
pret_materiel	BOOLEAN	Prêt de matériel inclus
restitution_materiel	BOOLEAN	Matériel restitué
mode_paiement	VARCHAR(50)	NOT NULL
statut_paiement	ENUM	{acompte_paye, paye}
montant_acompte	DECIMAL(10,2)	>= 0 (30% obligatoire)
acompte_paye_le	TIMESTAMP	Optionnel
statut	ENUM	10 valeurs possibles
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

Note v3 : Les attributs distance_km, prix_menu et prix_livraison sont conservés car nécessaires au calcul de tarification. Les champs paiement (mode_paiement, statut_paiement, montant_acompte, acompte_paye_le) et matériel (pret_materiel, restitution_materiel) ont été ajoutés. Le statut possède désormais 10 valeurs incluant deposit_pending et confirmed.

AVIS

ATTRIBUT	TYPE	CONTRAINTE
<i>id_avis</i>	UUID	Identifiant (PK)
note	INT	1 à 5
commentaire	TEXT	Optionnel
statut	ENUM	{pending, approved, rejected}
valide_par	UUID	FK vers UTILISATEUR (modérateur)
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

DEVIS

ATTRIBUT	TYPE	CONTRAINTE
<i>id_devis</i>	UUID	Identifiant (PK)
type_evenement	ENUM	{mariage, anniversaire, seminaire, cocktail, gala, autre}
date_evenement	DATE	NOT NULL
heure_evenement	TIME	Optionnel
adresse_evenement	TEXT	NOT NULL
ville_evenement	VARCHAR(100)	NOT NULL
nb_convives	INT	> 0
notes_regime	TEXT	Optionnel
message_client	TEXT	Optionnel
notes_internes	TEXT	Optionnel (visible employé/admin)
sous_total	DECIMAL(10,2)	>= 0
remise_pct	DECIMAL(5,2)	0 à 100
montant_remise	DECIMAL(10,2)	>= 0
total	DECIMAL(10,2)	>= 0
acompte	DECIMAL(10,2)	>= 0 (30% du total)
acompte_paye_le	TIMESTAMP	Optionnel
reference_acompte	VARCHAR(255)	Référence de paiement
date_validite	DATE	Optionnel
statut	ENUM	7 valeurs possibles
envoye_le	TIMESTAMP	Optionnel
accepte_le	TIMESTAMP	Optionnel
refuse_le	TIMESTAMP	Optionnel
instructions_acompte_envoyees_le	TIMESTAMP	Optionnel
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

LIGNE_DEVIS

ATTRIBUT	TYPE	CONTRAINTE
<i>id_ligne</i>	UUID	Identifiant (PK)
type	ENUM	{menu, option, prestation}
libelle	VARCHAR(200)	NOT NULL
prix_unitaire	DECIMAL(10,2)	>= 0
unite	ENUM	{par_personne, forfait, par_heure}
quantite	INT	> 0
total_ligne	DECIMAL(10,2)	>= 0
ordre_tri	INT	Ordre d'affichage
cree_le	TIMESTAMP	Défaut: maintenant

OPTION_DEVIS

ATTRIBUT	TYPE	CONTRAINTE
<i>id_option</i>	UUID	Identifiant (PK)
libelle	VARCHAR(120)	NOT NULL
description	TEXT	Optionnel
prix_unitaire	DECIMAL(10,2)	>= 0
unite	ENUM	{par_personne, forfait, par_heure}
actif	BOOLEAN	NOT NULL
cree_le	TIMESTAMP	Défaut: maintenant
modifie_le	TIMESTAMP	Mis à jour automatiquement

HORAIRE

ATTRIBUT	TYPE	CONTRAINTE
<i>id_horaire</i>	SERIAL	Identifiant (PK)
jour_semaine	INT	0 (dim) à 6 (sam), UNIQUE
heure_ouverture	TIME	Optionnel
heure_fermeture	TIME	Optionnel
ferme	BOOLEAN	NOT NULL

MESSAGE_CONTACT

ATTRIBUT	TYPE	CONTRAINTE
<i>id_message</i>	UUID	Identifiant (PK)
titre	VARCHAR(255)	NOT NULL
description	TEXT	NOT NULL
email	VARCHAR(255)	NOT NULL
lu	BOOLEAN	NOT NULL
lu_par	UUID	FK vers UTILISATEUR (employé/admin)
lu_le	TIMESTAMP	Optionnel
cree_le	TIMESTAMP	Défaut: maintenant

PAGE_LEGALE

ATTRIBUT	TYPE	CONTRAINTE
<i>id_page</i>	SERIAL	Identifiant (PK)
type	VARCHAR(50)	UNIQUE {mentions_legales, confidentialite, cgv}
titre	VARCHAR(255)	NOT NULL
contenu	TEXT	NOT NULL
modifie_le	TIMESTAMP	Mis à jour automatiquement

SECTION 03

Associations et cardinalités

Le MCD comporte 15 associations :

#	ASSOCIATION	ENTITÉ 1	CARD.	ENTITÉ 2	CARD.	DESCRIPTION
1	PASSER	Utilisateur	0,N	Commande	1,1	Un utilisateur peut passer 0 ou N commandes.
2	CONCERNER	Menu	0,N	Commande	1,1	Chaque commande concerne exactement 1 menu.
3	COMPOSER	Menu	1,N	Plat	0,N	Relation N,M via table menu_dishes.
4	ILLUSTRER	Menu	1,1	Image_Menu	0,N	Chaque menu est illustré par au moins 1 image.
5	RÉDIGER	Utilisateur	0,N	Avis	1,1	Un utilisateur peut rédiger 0 ou N avis.
6	ÉVALUER	Avis	1,1	Menu	0,N	Chaque avis évalue exactement 1 menu.
7	GÉNÉRER	Commande	0,1	Avis	1,1	Une commande peut générer 0 ou 1 avis.
8	DEMANDER	Utilisateur	0,N	Devis	1,1	Un utilisateur peut demander 0 ou N devis.
9	CONTENIR	Devis	1,N	Ligne_Devis	1,1	Un devis contient au moins 1 ligne.
10	RÉFÉRENCER	Ligne_Devis	0,1	Option_Devis	0,N	Une ligne peut référencer 0 ou 1 option.
11	INCLURE	Ligne_Devis	0,1	Menu	0,N	Une ligne de type menu peut inclure un menu.
12	CONVERTIR	Devis	0,1	Commande	0,1	Un devis accepté peut être converti en commande.
13	TRAITER	Utilisateur	0,N	Message_Contact	0,1	Un employé/admin peut traiter un message.
14	ASSIGNER	Utilisateur	0,N	Devis	0,1	Un employé peut être assigné à un devis.

15	MODÉRER	Utilisateur	0,N	Avis	0,1	Un employé/admin peut modérer un avis.
----	---------	-------------	-----	------	-----	--

Nouvelles associations v3 : #14 ASSIGNER (employé assigné au devis via `assigned_to`) et #15 MODÉRER (employé validant un avis via `validated_by`).

Cardinalités — notation synthétique

Relations principales

```

UTILISATEUR -(0,N)- PASSER      -(1,1)- COMMANDE
UTILISATEUR -(0,N)- DEMANDER    -(1,1)- DEVIS
UTILISATEUR -(0,N)- RÉDIGER    -(1,1)- AVIS
MENU         -(0,N)- CONCERNER  -(1,1)- COMMANDE
MENU         -(1,N)- COMPOSER   -(0,N)- PLAT
MENU         -(1,1)- ILLUSTRER  -(0,N)- IMAGE_MENU
AVIS         -(1,1)- ÉVALUER    -(0,N)- MENU
COMMANDE     -(0,1)- GÉNÉRER    -(1,1)- AVIS
DEVIS        -(1,N)- CONTENIR   -(1,1)- LIGNE_DEVIS

```

Relations optionnelles

```

DEVIS        -(0,1)- CONVERTIR  -(0,1)- COMMANDE
LIGNE_DEVIS  -(0,1)- RÉFÉRENCER -(0,N)- OPTION_DEVIS
LIGNE_DEVIS  -(0,1)- INCLURE     -(0,N)- MENU
UTILISATEUR  -(0,N)- TRAITER     -(0,1)- MESSAGE_CONTACT
UTILISATEUR  -(0,N)- ASSIGNER    -(0,1)- DEVIS
UTILISATEUR  -(0,N)- MODÉRER     -(0,1)- AVIS

```

SECTION 04

Règles de gestion

RG-01 — Gestion des rôles

Un utilisateur possède un rôle unique parmi : user (client), employee (employé), admin (administrateur). Le rôle détermine les droits d'accès dans l'application.

RG-02 — Composition des menus

Un menu est composé d'au moins un plat. Un plat peut appartenir à plusieurs menus. La relation est matérialisée par la table de jonction menu_dishes.

RG-03 — Commande liée à un menu

Chaque commande concerne exactement un menu. Le nombre de personnes doit être supérieur ou égal au minimum requis par le menu ($nb_persons \geq min_persons$).

RG-04 — Calcul du prix de commande

Le prix total d'une commande est calculé : $total = (prix_menu \times nb_personnes) + prix_livraison - remise$. La livraison est gratuite pour Bordeaux, sinon 5 € + 0,59 €/km.

RG-05 — Remise groupe

Une remise de 10 % est appliquée automatiquement si le nombre de convives dépasse le minimum requis de 5 personnes ou plus ($nb_persons \geq min_persons + 5$).

RG-06 — Gestion du stock

Le stock d'un menu est décrémenté atomiquement lors de la commande (UPDATE ... WHERE stock > 0). Si le stock est insuffisant, la commande est refusée (erreur 409).

RG-07 — Cycle de vie de la commande

Une commande suit un cycle de statuts ordonné :

```
en_attente → deposit_pending → confirmed → acceptee → en_preparation → en_livraison → livree →  
attente_retour_materiel → terminee
```

Elle peut être annulée à tout moment avant la livraison.

RG-08 — Acompte obligatoire

Un acompte de 30 % du montant total est obligatoire pour confirmer une commande ou un devis. Le solde de 70 % est dû le jour de la livraison.

RG-09 — Avis après commande

Un avis ne peut être déposé que pour une commande existante. Un seul avis par commande est autorisé. L'avis doit être modéré (pending → approved/rejected) avant publication.

RG-10 — Note de 1 à 5

La note d'un avis est un entier compris entre 1 et 5 inclus.

RG-11 — Structure d'un devis

Un devis contient au moins une ligne. Chaque ligne est de type menu, option ou prestation. Les lignes de type menu référencent un menu du catalogue, les lignes de type option référencent une option du catalogue.

RG-12 — Cycle de vie du devis

Un devis suit le cycle : draft → sent → accepted → acompte_paye → converti_en_commande. Il peut être refusé ou expirer à certaines étapes. Des instructions d'acompte peuvent être envoyées au client.

RG-13 — Conversion devis → commande

Un devis accepté (et acompte payé) peut être converti en une commande. La relation est optionnelle des deux côtés : un devis n'est pas forcément converti, une commande peut être passée sans devis.

RG-14 — Date de validité du devis

Un devis possède une date de validité. Passé cette date, le devis expire automatiquement si non accepté.

RG-15 — Types de plats

Chaque plat est classé dans une catégorie unique : entree, plat ou dessert.

RG-16 — Unités de tarification des options

Les options de devis sont facturées selon une unité : par_personne (multiplié par le nombre de convives), forfait (montant fixe) ou par_heure.

RG-17 — Consentement RGPD

L'inscription d'un utilisateur nécessite le consentement RGPD (rgpd_consent = true). La date de consentement est enregistrée. L'utilisateur peut demander l'anonymisation de ses données (Art. 17) et l'export de ses données (Art. 20).

RG-18 — Unicité des horaires

Chaque jour de la semaine (0 à 6) possède un unique enregistrement d'horaire. Un jour peut être marqué comme fermé.

RG-19 — Pages légales

Trois types de pages légales sont gérés : mentions_legales, confidentialite et cgv. Chaque type est unique.

RG-20 — Assignment des devis

Un devis peut être assigné à un employé (assigned_to). Cela permet le suivi et la gestion personnalisée des demandes clients.

RG-21 — Scoring client

Un score client est calculé : (+3 par devis accepté) + (-1 par devis refusé) + (+0,5 par tranche de 100€ dépensés). Ce score alimente le tableau de bord analytique.

RG-22 — Mots de passe

Les mots de passe doivent contenir au minimum 10 caractères, dont 1 majuscule, 1 minuscule, 1 chiffre et 1 caractère spécial. Ils sont hachés avec bcrypt (12 rounds).

SECTION 05

Correspondance MCD / Tables SQL

ENTITÉ MCD	TABLE SQL	CLÉ PRIMAIRE	REMARQUES
UTILISATEUR	users	id (UUID)	Rôles via ENUM user_role, index sur email
MENU	menus	id (UUID)	Filtrable par theme, diet
PLAT	dishes	id (UUID)	Type via ENUM dish_type
— (Composer)	menu_dishes	(menu_id, dish_id)	Table de jonction N,M
IMAGE_MENU	menu_images	id (UUID)	FK vers menus
COMMANDE	orders	id (UUID)	FK vers users, menus, optionnel quotes
AVIS	reviews	id (UUID)	FK vers orders, users, menus
DEVIS	quotes	id (UUID)	FK vers users, optionnel orders
LIGNE_DEVIS	quote_items	id (UUID)	FK vers quotes, optionnel menus, quote_options
OPTION_DEVIS	quote_options	id (UUID)	Catalogue de prestations
HORAIRE	business_hours	id (SERIAL)	Contrainte UNIQUE sur day_of_week
MESSAGE_CONTACT	contact_messages	id (UUID)	FK optionnelle vers users (read_by)
PAGE_LEGALE	legal_pages	id (SERIAL)	Contrainte UNIQUE sur type

Tables exclues du MCD (techniques / historique)

TABLE SQL	RAISON DE L'EXCLUSION
order_status_history	Table d'historique technique (logs de changements de statut des commandes)
quote_status_history	Table d'historique technique (logs de changements de statut des devis)
employee_contact_logs	Table de logs opérationnels (traçabilité interne des contacts client)

Ces tables sont nécessaires au fonctionnement de l'application mais ne représentent pas des entités métier conceptuelles. Elles sont générées automatiquement par les changements de statut et servent à la traçabilité.

SECTION 06

Corrections apportées (v3)

CORRECTION	AVANT (v2)	APRÈS (v3)	JUSTIFICATION
COMMANDE — attributs paiement	11 attributs, pas de gestion paiement	19 attributs avec paiement, matériel et acompte	Ajout mode_paiement, statut_paiement, montant_acompte, pret/restitution_materiel
COMMANDE — statuts	8 statuts	10 statuts (+deposit_pending, confirmed)	Gestion du flux d'acompte avant confirmation
DEVIS — attributs enrichis	14 attributs	22 attributs	Ajout assigned_to, message_client, notes_internes, timestamps envoi/accept/refus, instructions acompte
LIGNE_DEVIS — tri et FK	7 attributs, pas de FK explicites	9 attributs + FK menu_id, option_id	Ordre d'affichage (sort_order) et références directes menu/option
MESSAGE_CONTACT — traçabilité	5 attributs (lu boolean simple)	8 attributs + lu_par, lu_le	Qui a lu le message et quand
AVIS — modération	4 attributs	7 attributs + valide_par, timestamps	Traçabilité de qui a approuvé/rejeté l'avis
UTILISATEUR — RGPD	9 attributs	16 attributs complets	Ajout password_hash, reset_token, country, anonymized_at, date_consentement_rgpd, timestamps
Associations — 2 nouvelles	13 associations	15 associations	ASSIGNER (employé → devis) et MODÉRER (employé → avis)
Règles — enrichies	18 règles	22 règles	Ajout RG-08 acompte, RG-20 assignation, RG-21 scoring, RG-22 mots de passe
PAGE_LEGALE — types	{mentions, privacy, cgv}	{mentions legales, confidentialite, cgv}	Noms alignés avec le code réel